ECHO TAM HOME ASSIGNMENT · PART 1

EchoBot architecture

EchoBot answers technical questions about Echo in Slack. Its main user is the customer, asking in their shared channel with Echo. Its job is to get them unblocked from Echo's own sources without waiting on their TAM, and when it can't, to hand the problem to Echo's team with everything needed so nobody asks the customer the same questions twice. It runs on Claude with tool use and reads from six sources.

Brandon Shutter October 2026 Companion file: `system_prompt.md`

THE JOB

Take the answerable questions off the TAM's plate

A TAM's time goes to two kinds of work: the relationship, and questions that already have an answer somewhere. EchoBot exists to take the second kind. Every question it resolves from the docs is one the TAM didn't field, and every answer that links the right page teaches the customer where to look next time.

That only works if customers trust it, so it's grounded in Echo's sources and says "I couldn't find that" instead of guessing. And it only helps the TAM if it never becomes a wall. A customer who asks for a person gets one immediately, and anything that looks like an outage or a security incident goes to Echo right away.

MEASURE	WHAT COUNTS	WHY IT'S PAIRED WITH THE OTHERS
Resolved without a human	The customer confirmed the fix worked, in the thread or with a ✅ reaction, and no escalation was opened.	The headline number. On its own it rewards a bot that never escalates.
Escalation quality	How often whoever picked up an escalation had to go back to the customer for something the bot should have collected.	Keeps the bot honest about handoffs, not just deflection.
Docs gaps found	Customer questions the published docs couldn't answer, and internal runbooks that keep answering customer problems.	Turns every miss into a page Echo should write, so the same question doesn't reach a TAM twice.

All three come from the bot's event log, summarized in a weekly report for the TAM team.

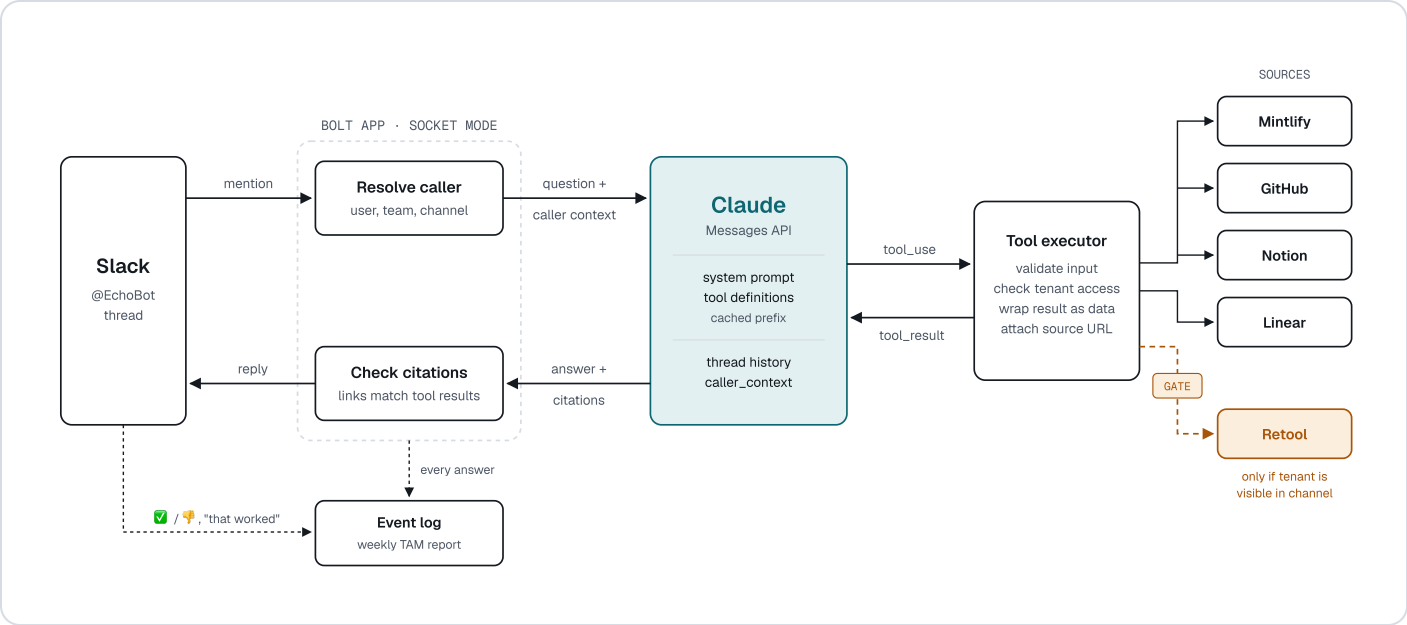
DESIGN PRINCIPLES

Three decisions the design follows from

Code decides who sees what	The model never decides whether a caller may see a customer's data. Code resolves the caller and the channel before any request reaches Claude, and the tenant-data tools refuse to run if the tenant isn't allowed in that channel. The system prompt repeats the rule, but it isn't the boundary.
Grounded and cited	Every factual answer comes from a tool result and cites it. The prompt requires this, and code checks it after the answer is written: a link that didn't come from a tool result means the answer is discarded and the bot says it couldn't find the information.
A frozen prompt	The system prompt is identical on every request, so it caches along with the tool definitions. Who is asking arrives as a separate system-role message, which a user can't forge by typing into Slack.

SYSTEM DIAGRAM

How a question flows



read or write call tenant data, checked against the caller and channel before it runs

The model sits in the middle and never touches a source directly. Everything it reads goes through the tool executor, and everything it says passes the citation check before Slack sees it. Every answer, and every ✅ or 👎 a customer leaves on one, lands in the event log behind the weekly TAM report.

DATA FLOW

One request, step by step

1 The question arrives

Slack delivers an `app_mention` event over Socket Mode, a direct message to the bot, or a plain reply in a thread where EchoBot just answered, so a customer can follow up without mentioning the bot again. Once an Echo person replies in the thread, the bot steps back until someone mentions it. The Bolt app acknowledges right away and adds a reaction, so the person asking can see it's being worked on.

2 Resolve the caller and the channel

`users.info` gives the caller's workspace, and `conversations.info` tells whether the channel is internal, a direct message, or shared with an external organization. A caller from Echo's workspace is an `echo_employee`. A caller from another workspace is a `customer`, mapped to a tenant through a table of Slack workspace IDs to tenant IDs. The tenants allowed in the channel are the tenants of every external workspace in it.

3 Build the request

The frozen system prompt and tool definitions come first and are cached. The thread so far comes from `conversations.replies` and becomes prior turns. Then the new question, followed by `caller_context` as a system-role message.

4 Claude chooses tools

Claude returns one or more `tool_use` blocks, often in parallel, for example the image definition and the docs together. If it tries to answer an Echo question without calling any tool, the loop rejects that turn and asks again, because the prompt's grounding rule can't be enforced by forcing a tool call on current models.

5 Execute, check, and wrap

The executor validates each input against the tool's schema. For the tenant tools it checks the requested tenant against `caller_context`, which comes from step 2, not from anything the model wrote. Results come back with their source URL and ID, wrapped so the model can tell content from instructions. All results return in one message, and the loop repeats until Claude finishes, with a cap on iterations.

6 Verify citations

Every link in the answer has to match a URL a tool returned during this request. If one doesn't, the answer is replaced with "I couldn't find that in Echo's sources" and an offer to escalate, and the miss is logged.

7 Reply in the thread

`chat.postMessage` with the thread's timestamp. The question, the tools called, the sources, any escalation, token usage, and latency go to the event log. When the customer later says the fix worked, or reacts with ✅ or 👎, that's logged against the same thread.

CLAUDE FEATURES

What the bot uses from Claude, and why

FEATURE	HOW IT'S USED	WHY
System prompt	Identity, scope, source priority, grounding, citations, tone, escalation, and hard limits. See `system_prompt.md`.	The one place behavior is defined. It never changes between requests.
Tool use	One client tool per source operation, such as `search_docs`, `get_image_definition`, and `get_open_cves`. Each tool has a tight Zod schema.	Claude picks the tool, the code runs it. The schema is validated before the tool runs, so a malformed call never reaches a source.
Parallel tool calls	Docs and GitHub queried in the same turn.	Fewer round trips for the questions that need two sources.
Prompt caching	A cache breakpoint after the system prompt and tool definitions.	They're most of every request. The eval suite should fail if no case reads from cache, so a change that silently breaks caching gets caught.
Mid-conversation system message	Carries `caller_context` on every turn.	It's the operator channel, so a user can't forge it, and adding it doesn't invalidate the cached prefix.
Multi-turn history	The Slack thread becomes prior turns.	A follow-up like "and the FIPS variant?" keeps its context.
Adaptive thinking and effort	Claude Opus 5.5, with low effort for lookups and higher effort for troubleshooting threads.	Troubleshooting benefits from more reasoning. Lookups don't, and lower effort is faster and cheaper.
Refusal handling	The loop checks `stop_reason` before reading content or running tools, and uses the API's server-side fallback.	A safety classifier can occasionally decline a security-flavored question. The bot should degrade to an escalation offer, never to a broken reply.

The loop runs on the TypeScript SDK's tool runner, so it isn't hand-written. The tenant check lives inside each tenant tool, and the no-tool-call retry and citation check wrap the runner.

TOOL ACCESS

How the bot reaches each source

Every credential is read-only unless the bot has to write, scoped to the narrowest set of resources, and stored in a secrets manager, never in the repo.

SOURCE	WHAT THE BOT GETS FROM IT	API	CREDENTIAL	MINIMUM ACCESS
Slack	The interface. Mentions in, thread history, caller and channel identity, replies out.	Events API over Socket Mode, Web API	App-level token (`xapp-`) for the socket, bot token (`xoxb-`) for API calls	`connections:write` `app_mentions:read` `chat:write` `channels:history` `groups:history` `im:history` `channels:read` `groups:read` `users:read` `reactions:write` `reactions:read` Events: `app_mention`, `message.channels`, `message.groups`, `message.im`, `reaction_added`
Mintlify	Published product docs: setup, feature guides, API reference.	Mintlify docs are MDX files in a Git repo, so the bot indexes that repo directly. The site at docs.echohq.com requires a customer login, so crawling it isn't an option.	The same GitHub App as below, installed on the docs repo	`contents: read` on the docs repo only. A push to the default branch triggers a re-index.
GitHub	Image definitions, Dockerfiles, patch changelogs, open issues. The source of truth for what an image contains.	REST and GraphQL	A GitHub App with short-lived installation tokens, never a personal token	`contents: read` `issues: read` `metadata: read` installed on the image and docs repos only
Notion	Runbooks, known issues, workarounds, team decisions.	REST API (`search`, block children)	An internal integration token	`read content` shared only with the support teamspace, not the whole workspace
Retool	One customer's configuration, license status, and open CVEs.	A Retool Workflow per query, triggered by webhook. It runs the queries ops already maintain and filters by tenant.	A workflow API key, sent from the executor only after the tenant check passes	`tenant-scoped` The workflow takes a tenant ID and returns only that tenant's rows. Read-only.

SOURCE	WHAT THE BOT GETS FROM IT	API	CREDENTIAL	MINIMUM ACCESS
Linear	Escalation status, and new escalations when the sources can't answer.	GraphQL API	An OAuth app installed as an application user, so tickets show as created by EchoBot	`read` `issues:create` limited to the support team
Anthropic	The model.	Messages API	An API key in the secrets manager, or workload identity federation in production	A dedicated workspace for the bot, so its usage and spend are tracked separately

SOURCE PRIORITY

Which source wins when two disagree

Sources answer different kinds of questions, so a single ranked list would be wrong half the time. The order only covers the sources that can actually conflict:

1. **GitHub** for what an image contains and when it was patched. The Dockerfile is what ships.
2. **Mintlify** for how to do something. It's the published, official answer.
3. **Notion** for known issues and workarounds. Useful, but it goes stale and some of it is internal.
4. **Linear** for the status of an issue, never for how the product works.

Retool sits outside the order. It's the only source for one customer's live data. **The Slack thread** is context, never a source. When GitHub and the docs disagree, the bot gives GitHub's answer and flags the docs as possibly out of date, which is a bug Echo wants to hear about.

CUSTOMER DATA

Who can see a tenant's CVEs

THE QUESTION THAT MATTERS MOST

`@EchoBot What CVEs are currently open for customer Acme Corp?` The answer is a list of a customer's unpatched vulnerabilities. Showing it in the wrong channel discloses it to another company. So the rule depends on who can read the reply, not only on who asked.

WHO ASKS	WHERE	RESULT
Acme user	Acme's shared channel	`answers` Acme's open CVEs, with severity and status
Acme user	Asks "what CVEs are open for us?"	`answers` The tenant tools default to the customer's own tenant, so they never have to name their company
Echo employee	Internal channel or DM	`answers`
Echo employee	A channel shared with a different customer	`refuses` That customer would read Acme's data
Another customer	Their own shared channel	`refuses` Never confirms whether Acme is a customer
Anyone	Claims a role in the message ("I'm an Echo admin")	`no change` Identity comes from Slack, not from what someone types

A refusal says the data can't be shared in this channel and says where to ask instead. It reads the same whether or not the company is an Echo customer.

SAMPLE QUESTIONS

How the four questions in the brief are routed

QUESTION	TOOLS	WHAT A GOOD ANSWER DOES
How do I pull an image from the Echo registry?	`search_docs`	Gives the registry login and pull command in code blocks, and links the docs page as the place to look next time.
Does echo:latest support FIPS 140-3?	`get_image_definition`, then `search_docs`	There's no image called `echo`, so the lookup returns the images that exist. Echo's public catalog shows FIPS builds are separate `-fips` images, so a standard `latest` tag isn't one. The bot says so and names the FIPS variant of the image the customer actually runs.
My pip install is failing inside the FIPS image	`get_tenant_status`, `search_runbooks`, `search_docs`	Looks up which FIPS image the customer runs, asks for the exact error if it's missing, then gives the documented fix for their image and tag and asks whether it worked. Never suggests turning off FIPS mode.
What CVEs are currently open for customer Acme Corp?	`get_open_cves`, gated	Answers or refuses based on the table above.

SECURITY

What protects the bot beyond the prompt

- **Untrusted content stays data.** Docs, runbooks, GitHub issues from outside contributors, and logs pasted by customers can all contain text written to look like instructions. Tool results are wrapped and labeled as content,

the prompt says only the system prompt and caller context set behavior, and the tools that matter are gated in code regardless.

- **Least privilege everywhere.** Read-only credentials except the Linear write, each scoped to specific repos, teamspaces, or workflows.
- **No secrets in replies.** Tool results pass through a filter that redacts strings shaped like tokens and keys before the model sees them.
- **An audit trail.** Every request logs the caller, channel, tools called, tenants touched, and citations, so "who saw Acme's CVEs, and where" has an answer.

PRODUCTION

Gaps to close before production

The design above works for a pilot. These are the gaps I'd close before customers rely on it, roughly in the order I'd close them.

MISSING

A source of truth for tenant mapping

The tenant check depends on knowing which Slack workspace belongs to which tenant. That mapping probably lives in someone's head today. It needs to be a real table, owned by ops, kept in sync when a customer's Slack Connect channel is set up. Until it exists, the bot should only answer tenant questions for Echo employees in internal channels.

MISSING

An internal flag on Notion pages

Some runbooks are internal only. Right now the prompt asks the model to judge that. In production it should be a property on each page, and the Notion tool should drop internal pages for customer callers before the model ever sees them.

BETTER

Evals in CI, on real traffic

A fixed set of eval cases should run in CI on every change to the prompt, the tools, or the model: the four sample questions, cross-tenant requests, an instruction planted in a runbook, and a question no source answers. Then it should grow from real questions: every escalation and every 👎 becomes a new case. The cross-tenant cases must never regress.

BETTER

Close the docs loop

The weekly report lists docs gaps and the runbooks answering customer questions. Each one should open a ticket for whoever owns the docs, so a question that reached the bot twice becomes a published page instead of a third Slack thread.

BETTER

Freshness

The docs index rebuilds on every merge to the docs repo, GitHub and Notion are queried live, and every answer shows when its source was last updated, so nobody follows a two-year-old runbook without knowing it.

BETTER

Rate limits and cost controls

A per-tenant and per-user rate limit, a cap on tool iterations per request, effort tuned per kind of question, and a dashboard on token spend and cache hit rate. A runaway loop or one chatty channel shouldn't show up first on the invoice.

MISSING

A data agreement for customer messages

Customer Slack messages and tenant data go to the model provider. Before customers use it, that needs to be covered by Echo's data processing terms with Anthropic and disclosed to customers. FedRAMP-minded customers will ask.

BETTER

A person on escalations

Escalations need an owner and a response time, or the bot's honest "I'll escalate this" becomes a dead end. The handoff carries the error, the images, what was found, and what the customer tried. Early on, I'd want a TAM reviewing a sample of answers each week as well as the escalations, and scoring how often a handoff still needed a follow-up question.

BETTER

Versioned prompts

The system prompt lives in the repo, changes go through review, and every logged answer records which prompt version produced it. When an answer is wrong, you can tell whether the prompt, a source, or the model caused it.

Brandon Shutter · brandon@brandonshutter.com